

SOLID Mapping

The SALESTORM architecture applies SOLID principles to keep the system modular, loosely coupled, testable, and easier to extend as new payment providers, infrastructure components, or business requirements are introduced.

1. Single Responsibility Principle (SRP)

Each service and component is given one clear responsibility.

- **Inventory Service** manages stock availability and reservations.
- **Payment Service** manages payment initiation and payment status.
- **Order Service** manages the order lifecycle.
- **Notification Service** handles customer notifications.
- **Idempotency Service** handles duplicate-request detection.

For example, `ReservationService` is responsible only for creating, confirming, expiring, and releasing reservations. It does not contain payment or order-creation logic.

This keeps individual components focused and prevents one service from becoming responsible for the entire purchase workflow.

2. Open/Closed Principle (OCP)

The payment layer is designed so that new payment providers can be added without modifying the existing payment business logic.

A common `PaymentGateway` interface defines the required payment operations.

Possible implementations include:

- `RazorpayAdapter`
- `StripeAdapter`
- `MockGatewayAdapter`

`PaymentService` depends on the `PaymentGateway` abstraction rather than a specific provider.

Therefore, if a new payment provider is introduced, a new adapter can be added without changing the core `PaymentService` logic.

Evidence:

PaymentGateway → RazorpayAdapter / StripeAdapter / MockGatewayAdapter

3. Liskov Substitution Principle (LSP)

All implementations of PaymentGateway must follow the same contract expected by PaymentService.

Each implementation must support:

- Idempotent charge() operations.
- Payment status lookup through getStatus().
- Bounded request timeouts.
- Consistent handling of successful, failed, and uncertain payment results.

For example, MockGatewayAdapter can replace RazorpayAdapter during testing without requiring any changes to PaymentService.

The calling service should not need to know which payment provider is being used.

Evidence:

PaymentService → PaymentGateway → interchangeable gateway implementations.

4. Interface Segregation Principle (ISP)

SALESTORM uses small, focused interfaces instead of large interfaces containing unrelated operations.

Examples include:

- PaymentGateway
- InventoryRepository
- EventPublisher
- IdempotencyStore

A component depends only on the operations it actually requires.

For example, the **Order Service** does not need access to payment-provider operations. It receives the required payment-success event and performs its own order-related responsibilities.

Similarly, the **Payment Service** does not need direct access to inventory implementation details.

This keeps service boundaries clear and reduces unnecessary dependencies.

5. Dependency Inversion Principle (DIP)

Core business logic depends on abstractions rather than directly depending on infrastructure technologies. For example:

PaymentService

→ PaymentGateway

→ RazorpayAdapter / StripeAdapter

The payment business logic therefore does not depend directly on Razorpay or Stripe.

Similarly:

InventoryService

→ InventoryRepository

→ SQLInventoryRepository

And:

OrderService

→ EventConsumer

→ Kafka implementation

The database, message broker, and payment provider are treated as implementation details behind defined interfaces. Dependencies are injected into the services, allowing implementations to be replaced or mocked without changing the core business logic.

SOLID in the SALESTORM Architecture

Together, these principles provide the following benefits:

- **Clear responsibilities** across services and components.
- **Loose coupling** between business logic and infrastructure.
- **Easy testing** using mock implementations.
- **Safe extension** when adding payment providers or infrastructure components.
- **Reduced impact of technology changes** on the purchase workflow.
- **Better maintainability** as the system evolves.

The key design goal is to keep the **purchase, inventory, payment, and order business logic independent from specific infrastructure implementations**, while allowing those implementations to be replaced or extended when required.